

FEM Homework

Jules Herrmann

28. March 2026

In this report, I explain my work for the homework of FEM class.
I chose to work on Project 4 : Elimination tree and direct sparse Cholesky solver.
I also implemented another sphere mesh, based on the geodesic subdivision of an icosahedron.

1 Sparse Cholesky Decomposition

This work follows the method proposed in J. Scott and M. Tuma [1].

In the following I use A^a for the “anti-transpose” of the matrix A . That is, the matrix obtained by mirroring along the anti-diagonal, or equivalently the matrix $\tilde{I}A^T\tilde{I}$ where $\tilde{I}$ is a matrix with only ones on the antidiagonal.

1.1 Symbolic phase

Function `void elimination_tree(const CSRMatrix &A, TArray<uint32_t> &parent)` is my implementation of algorithm 4.2 of [1].

The pseudo code uses an indexing scheme starting at 1 and uses 0 to denote the absence of a parent. Since I transcribe the algorithm to use 0-based indexing, I chose to denote the absence of parent with the maximum integer `UINT32_MAX`.

Function `cholesky_sparsity_pattern` is my implementation of algorithm 4.3 of [1].

If A is the input sparse matrix, and LL^T its Cholesky factorization, then this function returns both the sparsity pattern of L and L^a . It proceeds by creating the graph of L in the adjacency list data structure (resp. `rowL` and `colL`). It fills it in according the algorithm, then build the sparsity patterns.

1.2 Sparse triangular solver

Let's first describe the data structure used to store a sparse vector.

Let v be a vector with `nnz` non-zero coefficients. This vector is described with a buffer `v` containing contiguously all the non-zero coefficients. A second buffer `v_ind` of type `uint32_t *` contains the index of the non-zero coefficients in such a way that if $v_i = \alpha$, then there exist a k such that $v[k] = \alpha$ and $v_ind[k] = i$. Finally `v_size` stores `nnz`.

Two functions `sparse_tri_solve` have been implemented. One deals with dense vectors and a sparse matrix, while the other deals with sparse vectors.

The algorithm is a simple forward substitution algorithm, where, at step `row`, we compute

$$x_{\text{row}} = \frac{b_{\text{row}} - \sum_{\text{col}=1}^{\text{row}-1} L_{\text{row,col}} x_{\text{col}}}{L_{\text{row,row}}} \quad (1.1)$$

The only difficulty is the computation of the sum in the case of sparse vectors. We want to iterate over x and a row of L , which are both sparse vectors, so some care is required for the iteration to stay synchronized.

1.3 Numerical phase

The numerical phase of the Cholesky factorization uses the *up-looking* cholesky algorithm.

Indeed this algorithm can be implemented in such a way that the only iterations over consecutive values of the matrix happen along a row. Indeed, I use the Compressed Sparse Row (CSR) scheme to store my matrices and the scheme doesn't allow for fast iterations over a column.

Up-looking Cholesky factorization

```

1  $L_{11} \leftarrow (A_{11})^{\frac{1}{2}}$ 
2 for  $i$  from 2 to  $N$ 
3   | Solve  $L_{1:i-1,1:i-1} z = A_{i,1:i-1}$ 
4   |  $L_{i,1:i-1} \leftarrow z$ 
5   |  $L_{i,i} \leftarrow (A_{ii} - z z^T)^{\frac{1}{2}}$ 
6 end
```

This method uses the triangular solver that I presented in Section 1.2.

Since the rows of A , as well as their indices, are stored contiguously in memory, no data movement is required to call the sparse triangular solver at line 3. I just needed to compute the size of the sparse vector, that is, the number of non-zero coefficients of index inferior to i , which I do in linear time.

Function `sparse_cholesky` is my implementation of the algorithm.

1.4 Sparse Cholesky solver

I created a `CholeskySolver` struct that contains the lower subdiagonal matrix L , as well as L^a .

L^a is also lower subdiagonal, this way, it can be used with the forward substitution algorithm I implemented.

L^a can be used to solve linear problem with L^T in the following way :

$$L^T x = b \Leftrightarrow \tilde{I} L^T \tilde{I} x = \tilde{I} b \Leftrightarrow L^a x^r = b^r \quad (1.2)$$

where $v^r = \tilde{I} v$ is the vector in reversed order.

Therefore the following algorithm is used to solve $Ax = b$

Cholesky solver

- 1 Solve $Ly = b$
 - 2 Solve $Lz = y^r$
 - 3 return z^r
-

Method `CholeskySolver::solve` implements this algorithm.

A method `CholeskySolver::update_same_pattern` has also been implemented which allows to update the coefficients of the solver without recomputing the sparsity patterns. This is required because the interface of the demonstration program allows the user to change the value of dt and ν .

2 Better Sphere mesh

3 Code Structure

4 Build Instructions

Bibliography

- [1] J. Scott and M. Tůma, *Algorithms for Sparse Linear Systems*. in Nečas Center Series. Springer International Publishing, 2023.